SW Engineering CSC648-848 Fall 2025

Project Tutorall, Milestone 4

Team 4
Matthew Davis (Team Lead) - mdavis09@sfsu.edu
Tan Huynh (Back End Lead)
Anmol Tadikonda (Github Master)
Zainab Abbasi (Front End Lead)
Justine Tenorio

History	
Date Submitted	12/17/2025

1) Product summary

The app is named TutorAll.

TutorAll is an SFSU specific tutoring website. Students can find tutors specific to classes. Usually, tutoring is hard to get, as office hours often conflict with classes and third party tutoring is expensive and harder to use. TutorAll streamlines the tutor finding process.

- ★ Search by major and course
- ★ Search by professor and sort by price
- ★ See about page, including developer info
- ★ Fill out a form to become a tutor
- ★ See your profile and your listings
- ★ See messages and sent one-way messages
- ★ Create listing and see their status
- ★ The website adjusts layout depending on website dimensions

The IP address is 3.216.62.166

2) Usability test plan for selected function

The objective of this usability test is to evaluate how easily beginners can navigate TutorAll, search tutor by major and class, and search by name and sort by price. The test aims to identify navigation struggles, convoluted UI, and issues preventing accomplishing the task. Findings will be used to simplify user flows and improve UI.

1) The tester needs a laptop or computer with a stable internet connection.
2) The tester needs to open Docker
3) The tester needs to go into the TutorAll project via terminal. Then go to the application directory
4) docker compose -f compose.dev-backend.yml down --remove-orphans
5) docker compose -f compose.dev-backend.yml up --build
6) Go to IP address 3.216.62.166
7) Will land on the website at the home page /

The intended users are SFSU students. No prior experience is needed, especially as real users would begin with no experience.

The URL is 3.216.62.166.

The test occurs remotely, so can occur at home or anywhere with a laptop or computer. The test is asynchronous. The test will not be monitored, rather, as the test occurs, feedback will be noted. No training is needed before testing.

1) Please filter the tutors to search for by major.
2) Please search by class
3) Please complete the search and get to the search results
4) Please add a professor to the search query
5) Please sort by price

Effectiveness would be measured by how long does it take to figure how to complete tasks, does the tester express any confusion, how much assistance is required, does the user immediately know the search results refreshed.

Efficiency will be measured by number of clicks, is it as easy as it should be, the amount of time needed

- I could figure out how to complete the tasks
- I knew when the search refreshed
- Searching is beginner friendly

3) QA test plan and QA testing

The search function is being tested, including, but not limited to, the ability to provide search results, the accuracy of the search results, if the database works, will be tested.

1) The tester needs a laptop or computer with a stable internet connection.
2) The tester needs to open Docker
3) The tester needs to go into the TutorAll project via terminal. Then go to the application directory
4) docker compose -f compose.dev-backend.yml down --remove-orphans
5) docker compose -f compose.dev-backend.yml up --build
6) Go to IP address 3.216.62.166
7) Will land on the website at the home page /

Search feature is the feature to be tested

Test #	Test Title	Test Description	Test Input	Expected Correct Output	Test Results
1	Search by	Search only	Physics	Two tutors	

	major	by the drop down with majors		with PHYS classes	
2	Search by class	Search only by class with text input	CSC 648	One tutor with CSC 648 class	
3	Search major and class	Search with both major and class	Major: Mathematics Class: MATH 324	Two tutors with MATH 324 class	
4	Sort by price	Sort listings by price	Descending	More expensive tutor is first, less expensive tutor is last	

4) Peer Code Review:

Outlook

CSC 648 Code Review

From Matthew Allan Davis <mdavis9@sfsu.edu>
Date Sun 12/14/2025 5:29 PM
To Anmol Tadikonda <atadikonda@sfsu.edu>

Hello Anmol,

As our M4 QA and usability tests are on Search, it is important to do a full review of a substantial part of the Search code.

Please review the primary Advert Return (homesearch.json) component in Github through Github Review. This can be done by going to https://github.com/CSC-648-SFSU/csc648-fa25-03-team04/pull/65 and navigating to Files Changed.

You are required to check for basic header and in-line comments, ensure class/method/variable names are consistent with our naming schema and data variables, commit comments (all commit messages for file can be found here https://github.com/CSC-648-SFSU/csc648-fa25-03-team04/commits/development/application/frontend/tutorall/src/common/homesearch/searchAdverts.js), and general readability.

Please also give any additional comments and a summary of the code review as a reply to this email.

Complete this at your earliest convenience.
Thank you for your time,
Matthew A. Davis

AT

Anmol Tadikonda

To: Matthew Allan Davis

Reply Reply all

Hi Matthew,

Will do! Thanks for the links!

Best,
Anmol Tadikonda

...

AT

Anmol Tadikonda

To: Matthew Allan Davis

Reply Reply all

Hi Matthew,

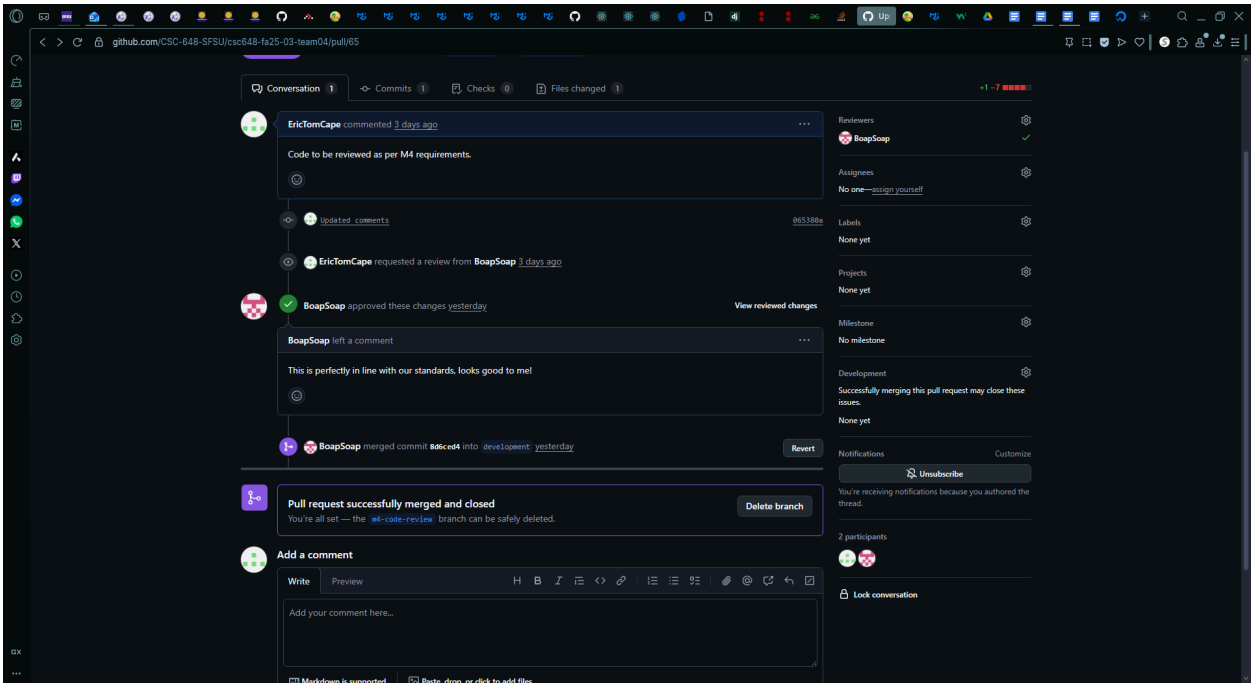
Sorry forgot to add the comments!

Checked everything locally and through Github's site everything looks good to me and follows our standards for commit comments etc.

Best,
Anmol T.

...

Reply Forward



```
export const SearchAdverts = () => {
  const queryClient = useQueryClient()
  const client = getPublicFetch()
  let params = useParams()

  //Search and sort parameters
  const [categories, setCategories] = React.useState(params.categories ? params.categories : "")
  const [courses, setCourses] = React.useState(params.courses ? params.courses : "")
  const [professors, setProfessors] = React.useState("")
  const [sort, setSort] = React.useState("")
  const priceSort = ["Ascending", "Descending"]

  //internal order value using static price sort requirements
  const handlePrice = (e) => {
    if (e.target.textContent == "Ascending") {
      setSort("")
    } else if (e.target.textContent == "Descending") {
      setSort("-")
    } else {
      setSort("")
    }
  }

  const handleProfessors = (e) => {
    setProfessors(e.target.value)
  }

  //Render option if sort/search returns no adverts
  const noProfessors = (data) => {
    if (data.length == 0) {
      return(
        <Grid size={12}
          sx={{
            flexGrow: 1,
            justifyContent: 'flex-start',
            alignItems: "center",
            display: "flex"
        }}>
          <Box sx={{
            flexGrow: 1,
            justifyContent: 'flex-start',
            alignItems: "center",
            display: "flex"
          }}>
            <Typography variant="body1"> It looks like we can't find any tutors that match your search! Please try again at a later time, or perhaps consider applying as a tutor yourself! </Typography>
          </Box>
        </Grid>
      )
    }
  }
```

```
  return (
    <Box sx={{ flexGrow: 1, display: 'flex', flexDirection: 'column'}}>
      <Grid container spacing={3}   sx={{
      justifyContent: "space-around",
      alignItems: "flex-start",
      flexGrow: 1
    }} >
      <Grid size={12} >
        {/*Paper provides a top bar with additional sort/order requirements*/}
        <Paper sx={{ width: 'calc(100% - 10px)', height:75, bgcolor: 'secondary.main', display: "flex", alignItems: "center", justifyContent: "center", paddingLeft: "5px", paddingRight: "5px", }}>
          <Grid container spacing={3} sx={{
            justifyContent: "space-around",
            alignItems: "center",
            flexGrow: 1
          }}>
            <Grid size={3}>
              <Typography> Showing {data.length} results. </Typography>
            </Grid>
            <Grid size={3}>
              <TextField
                onChange={handleProfessors}
                defaultValue={professors}
                placeholder="Search by Professors"
                label="Professors"
                />
            </Grid>
            <Grid size={3}>
              <Autocomplete
                disablePortal
                options={priceSort}
                sx={{ width: 200 }}
                onChange={handlePrice}
                renderInput={(params) => <TextField {...params} defaultValue={sort} label="Sort by Price Range" variant="standard" />}
                />
            </Grid>
            <Grid size={3}>
              <Button onClick={() => queryClient.invalidateQueries({ queryKey: ['adverts']})} >
                Apply
              </Button>
            </Grid>
          </Grid>
        </Paper>
      </Grid>
      {/*Render all returned adverts*/}
      {data?.map((advert) => (
        <Grid size={{ sm: 4, md: 2 }}>
          <AdvertItem advert={advert} />
        </Grid>
      ))}
      {noProfessors(data)}
      </Grid>
    </Box>
  )
}
```

5) Self-check on best practices for security – ½ page

Asset to be protected	Types of possible/expected attacks	Consequence of security breach	Your strategy to mitigate/protect the asset
User Database	SQL Injection, Unauthorized user access	High - User data may be lost, trust will be lost, admin access would allow authorization of media against ToS	Password encryption, database backups, field sanitization, authorization restrictions on certain models, phishing training, advert verification and change monitoring
Media Filesystem	Unauthorized user access	Medium - User files can be stolen or deleted	Password protection, data backup, file encryption

For field validation and sanitization, we use YUP (a field validator library for react) to control form fields, and django has inbuilt field sanitization to prevent SQL injection. This protects all forms from SQL injection and protects the search field as well.

Screenshot of password encryption:


```
{
    "first_name": "Alice",
    "last_name": "Smith",
    "email": "alice.smith@example.com",
    "password": "pbkdf2_sha256$1200000$habQJ0oqdyVEZeMnxY6mFK$dzQyxllzjSCQjOZBPCvtooOUSHn3LfUBTZRuIExKf+Q=",
    "is_active": true
},
```

6) Self-check of the adherence to original Non-functional specs – performed by team leads

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0 - **DONE**
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers - **DONE**
3. All or selected application functions shall render well on mobile devices (no native app to be developed) - **ON TRACK**
4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students - **DONE**
5. Critical data shall be stored in the database on the team's deployment server. - **DONE**
6. No more than 50 concurrent users shall be accessing the application at any time - **ON TRACK**
7. Privacy of users shall be protected - **DONE**
8. The language used shall be English (no localization needed) - **DONE**
9. Application shall be very easy to use and intuitive - **DONE**
10. Application shall follow established architecture patterns - **DONE**
11. Application code and its repository shall be easy to inspect and maintain - **DONE**
12. Google analytics shall be used - **ON TRACK**
13. No e-mail clients shall be allowed. Interested users (clients) can only message to service providers via in-site messaging. One round of messaging (from client to servcie provider) is enough for this application. No chat functions shall be developed or integrated - **DONE**
14. Pay functionality (e.g. paying for goods and services) shall not be implemented nor simulated in UI. - **DONE**
15. Site security: basic best practices shall be applied (as covered in the class) for main data items - **DONE**
16. Media formats shall be standard as used in the market today - **ON TRACK**
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools - **ON TRACK**
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only" at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application). - **DONE**

7) Use of genAI tools like ChatGPT and copilot (mandatory)

The GenAI tool used is ChatGPT Edu. It was HIGHly useful. It was used for clarifying how to create the usability test plan and QA test plan, as well as suggesting improvements. Used to provide feedback and clear up confusion